The DOM Serialization Gap: Architectural Pruning, Information Loss, and Observation Confounds in Autonomous Browser Agents

A Systems Audit and Case Study of browser-use 0.13.10

Anonymous Authors
Department of Computer Science
`Paper Under Double-Blind Peer Review`

October 2026

Abstract

Autonomous web agents powered by Large Language Models (LLMs) are increasingly evaluated on multi-step transactional workflows. While existing benchmarks evaluate end-to-end task completion, the client-side Document Object Model (DOM) observation pipelines that prune and serialize web markup prior to model ingestion remain unexamined. In this paper, we conduct an empirical systems audit of the DOM serialization gap—the structural divergence between live browser representations and the serialized text ingested by LLMs—focusing on `browser-use 0.13.10` under text-only observation as a representative case study. Through static AST analysis and dynamic execution traces, we characterize three deterministic pruning bottlenecks that exclude structured Schema.org JSON-LD, strip non-interactive container WAI-ARIA labels, and omit custom developer `data-*` attributes. Comparing the agent's selector map against native Chromium DevTools accessibility trees, we document that 27 of 34 interactive elements (79.4%) match Blink accessibility nodes on the tested route. Furthermore, we identify the Action-Label Co-Location Confound, an exploratory cloud pilot ceiling effect, and analytical sample-size boundaries demonstrating that small-sample evaluations ($n \leq 20$ per arm) lack statistical power ($\leq 11\%$) to detect moderate effect sizes.

Keywords: Autonomous Web Agents, Large Language Models, DOM Serialization Gap, WAI-ARIA, Semantic Markup, Accessibility Trees, Experimental Reproducibility.

1 Introduction and Research Motivation

The integration of Large Language Models (LLMs) with automated browser execution environments has catalyzed the emergence of autonomous web agents capable of executing multi-turn transactional workflows across digital interfaces [3–5, 15, 18]. Operating as sequential decision-making systems, these agents observe web page state, synthesize an action plan, and dispatch discrete interactions (e.g., clicking elements, typing text, selecting options) via browser automation protocols such as Playwright, Puppeteer, or the Chrome DevTools Protocol (CDP).

Despite widespread empirical benchmarking, contemporary web agent research suffers from a pervasive methodological confound: evaluations are typically conducted holistically across diverse web environments where visual styling, page layout, JavaScript routing logic, and DOM markup representations vary simultaneously. On complex websites, it is difficult to disentangle whether an agent failed due to visual occlusion, context saturation, stochastic planning drift, or uninformative DOM node labeling.

In modern frontend web engineering, information is intentionally partitioned into distinct architectural layers:

- **Rendered Visible Typography:** Plain textual nodes rendered inside standard HTML tags (e.g., `<h1>`, `<p>`, `<span>`) intended for human visual reading.
- **Semantic Accessibility Markup (WAI-ARIA):** Programmatic semantic annotations (e.g., `role`, `aria-label`, `aria-describedby`) standardized by the World Wide Web Consortium [13, 14] to convey interactive semantics to assistive technologies.
- **Developer Custom Data Attributes:** Structured key-value pairs (e.g., `data-action`, `data-product-id`, `data-category`) utilized by client-side Single Page Applications (SPAs) for internal state management.
- **Structured Metadata (Schema.org JSON-LD):** Standardized machine-readable entity graphs embedded inside non-rendered script tags (`<script type="application/ld+json">`) for search engine indexing [12].

As industry discussions propose modifying web architectures for autonomous agents—including proposals to enrich ARIA labels specifically for AI scrapers or to deploy dedicated machine endpoints (e.g., `llms.txt` [8], Model Context Protocol APIs [2])—a fundamental systems question arises:

> *What information does a browser-agent observation pipeline preserve or discard between webpage markup, browser representations, and the final model observation?*

If an agent framework silently purges structured metadata, web authors' efforts to markup pages for AI agents are lost. Furthermore, if researchers benchmark agent performance across markup variants without verifying prompt delivery, they risk evaluating an unaugmented text baseline while mistakenly attributing performance variations to invisible channels.

In this paper, we address this question through an in-depth empirical systems audit of `browser-use 0.13.10` under text-only observation (`use_vision=False`). We treat `browser-use 0.13.10` as a focused single-framework case study. Our contributions are strictly scoped to what our verified evidence establishes:

1. **Source-Level Characterization:** We identify three deterministic pruning heuristics in `browser-use 0.13.10` that control metadata filtering and serialization.
2. **Reproducible Reachability Tracing:** We trace information retention across five distinct pipeline representations, demonstrating that Schema.org JSON-LD and container ARIA attributes are excluded from the final observation stream under default configurations.
3. **Comparative Accessibility Audit:** We compare the agent's interactive selector map against Chromium's native accessibility tree, reporting a 27/34 (79.4%) correspondence census on the tested store route and characterizing the structural divergence between native accessibility caches and heuristic DOM serializers.
4. **Experimental Confound and Power Analysis:** We formalize the Action-Label Co-Location Confound, disclose ceiling effects in an exploratory 41-trial cloud pilot, and calculate the sample-size boundaries required for valid equivalence testing in future web agent ablation benchmarks.

2 Related Work and Systems Framing

2.1 Autonomous Web Agent Benchmarks

Evaluation of LLMs as autonomous web agents has advanced through several benchmark suites:

- **WebArena** [18] established an end-to-end evaluation paradigm across four realistic self-hosted web applications, revealing task completion rates between 10.6% and 14.4%. However, WebArena evaluates holistic task success across multi-page workflows, conflating layout design, visual rendering, and markup structure.
- **Mind2Web** [4] introduced a dataset of 2,350 tasks across 137 real websites, evaluating element-level action prediction offline from DOM tree snapshots and demonstrating the severity of the raw HTML context bottleneck.
- **WorkArena and BrowserGym** [3,5] evaluated web agents on enterprise workflows within ServiceNow. To mitigate context window exhaustion, BrowserGym utilizes Accessibility Trees (AXTrees), condensing long raw HTML documents into compact interactive token sequences. BrowserGym treats accessibility trees as an operational engineering optimization.
- **OSWorld** [15] extended multimodal agent evaluation to open-domain operating system tasks across heterogeneous applications, illustrating that execution grounding bottlenecks persist across diverse interface modalities.
- **WebAgent** [6] addressed HTML length bottlenecks by pre-training a specialized model to sub-sample and filter long documents prior to action planning.
- **AgentBench** [10] evaluated LLMs across eight interactive environments, confirming substantial performance drops on web tasks relative to static reasoning benchmarks.

2.2 DOM Observation Pruning and Distillation

A critical challenge in web agent engineering is observation-space reduction:

- **AgentOccam** [16] demonstrated that aggressive observation-space pruning yields a strong baseline for web agents, showing that reducing DOM size mitigates context bloat. However, AgentOccam focused on operational token reduction rather than auditing whether heuristic pruning selectively purges semantic metadata channels.
- Heuristic tag and attribute whitelisting in frameworks such as `browser-use 0.13.10` attempts to remove non-essential markup while preserving clickable elements. However, the precise information-loss characteristics of these heuristics have not previously been decomposed across frontend metadata channels.

2.3 Multimodal Visual Grounding vs. Text Observation

A parallel research line integrates computer vision with web navigation:

- **VisualWebArena** [9] extended WebArena to visually intensive tasks, showing that text-only agents fail when critical information is encoded exclusively in pixels.
- **SeeAct** [17] formulated a two-stage visual agent framework separating candidate generation on screenshots from action grounding on DOM elements.
- **WebVoyager** [7] evaluated multimodal agents using Set-of-Mark visual prompts across 15 live websites, achieving 55.7% completion. Visual screenshot prompts bypass DOM observation pipelines entirely, trading structured machine semantics for pixel rendering.

2.4 The Systems Audit Contribution

Existing literature addresses task complexity, visual grounding, and model capabilities. However, prior studies have not systematically audited the client-side DOM serialization algorithms in open-source agent libraries to measure what information is preserved or discarded between source HTML, live DOM trees, browser accessibility trees, and final LLM prompt buffers. While auxiliary channels (such as page titles, current URL, active JavaScript execution tools, or visual screenshots) may supply additional context, the text-based DOM observation buffer remains the primary grounding mechanism for text-based browser agents.

3 Systems Architecture and Pruning Heuristics in `browser-use 0.13.10`

We performed a static AST source-code audit and runtime execution trace of `browser-use 0.13.10` (PyPI binary release wheel distribution). The framework processes live web pages by walking the Chromium DOM tree and serializing it into a text string for the language model. We identified three deterministic pruning mechanisms:

3.1 Script and Meta Pruning (`DISABLED_ELEMENTS`)

In `browser_use/dom/serializer/serializer.py` (Line 18), the framework defines an explicit exclusion set:

```
DISABLED_ELEMENTS = {'style', 'script', 'head', 'meta', 'link', 'title'}
```

During simplified tree creation (`_create_simplified_tree`, Lines 480–482):

```
elif node.node_type == NodeType.ELEMENT_NODE:
    if node.node_name.lower() in DISABLED_ELEMENTS:
        return None
```

Any element matching `DISABLED_ELEMENTS` is immediately pruned from the observation graph; child nodes are never traversed. Consequently, `<script type="application/ld+json">` structured data and `<meta>` tags are completely excluded from the serialized observation tree under this serialization path.

3.2 Non-Interactive Element Tag Stripping

In `browser_use/dom/serializer/serializer.py` (Lines 1046–1058), element tag serialization is conditionally gated:

```
is_any_scrollable = node.original_node.is_actually_scrollable or node.original_node.
    is_scrollable
if (
    node.is_interactive
    or is_any_scrollable
    or node.original_node.tag_name.upper() == 'IFRAME'
    or node.original_node.tag_name.upper() == 'FRAME'
):
    next_depth += 1
    attributes_html_str = DOMTreeSerializer._build_attributes_string(
        node.original_node, include_attributes, text_content
    )
```

If an element is **not interactive** (not clickable, not scrollable, and not an iframe):

- `_build_attributes_string` is never executed.

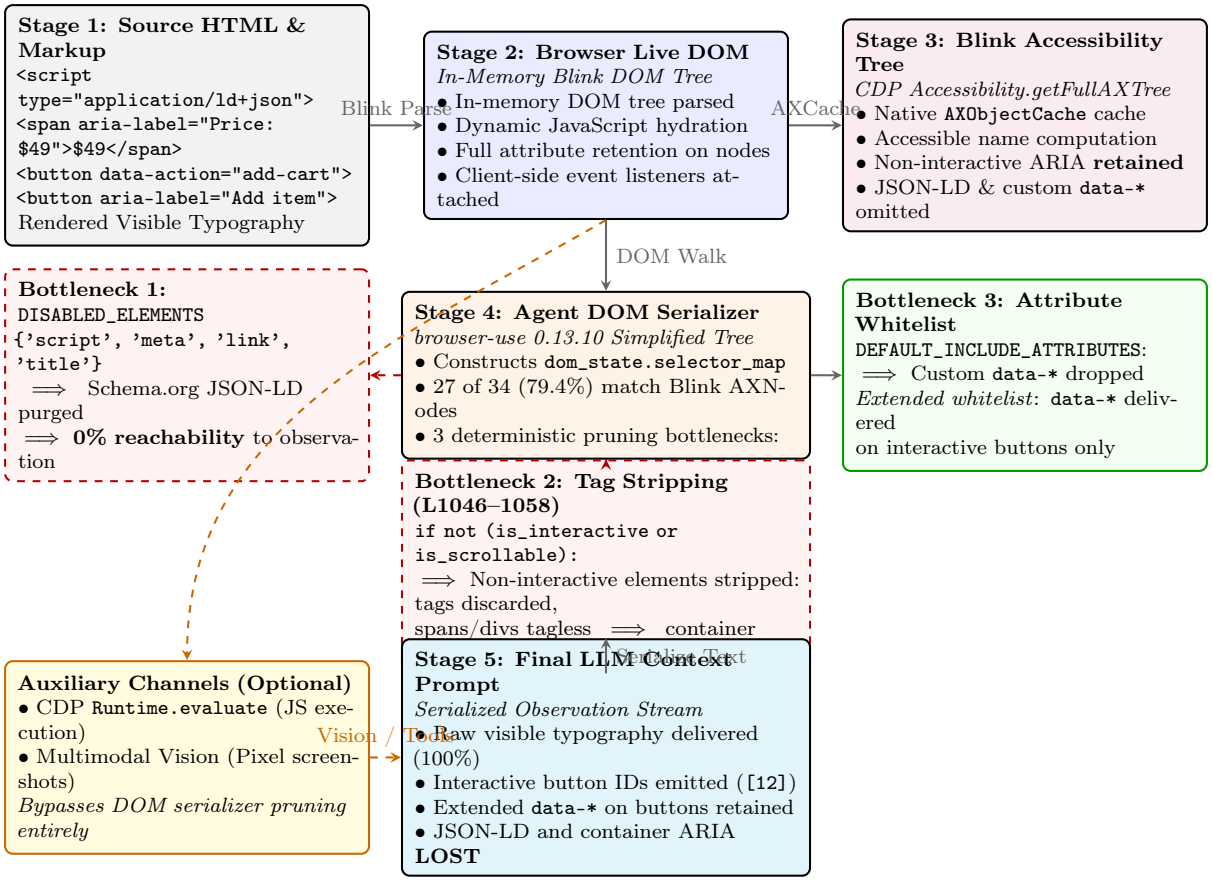


Figure 1: End-to-end architecture of the autonomous browser agent observation pipeline (`browser-use 0.13.10`). Web markup transitions across five discrete representations from source HTML to final LLM prompt. Three deterministic pruning bottlenecks in Stage 4 selectively purge structured Schema.org JSON-LD, strip non-interactive WAI-ARIA annotations, and filter custom developer attributes. Auxiliary channels (CDP JavaScript evaluation or visual screenshots) bypass serializer pruning entirely.

- No opening or closing tag is output for that element.
- The serializer processes child text nodes, emitting raw strings, but all element attributes (`aria-label`, `data-*`, `role`) on container tags (such as `<span>` and `<div>`) are discarded.

3.3 Attribute Whitelist Filtering

Attribute retention on interactive elements is governed by `DEFAULT_INCLUDE_ATTRIBUTES` (`browser_use/dom/vie` Line 18):

```
DEFAULT_INCLUDE_ATTRIBUTES = [
    'title', 'type', 'checked', 'id', 'name', 'role',
    'value', 'placeholder', 'alt', 'aria-label', ...
]
```

While standard accessibility attributes (`aria-label`, `role`) are included, developer custom attributes (`data-action`, `data-product-id`) are omitted by default. In `serializer.py` (Lines 1211–1215), attributes are discarded if `key not in include_attributes`. When runner configurations pass an extended whitelist (`EXTENDED_ATTRIBUTES`), custom attributes are retained on interactive nodes (e.g., buttons), but remain stripped on non-interactive containers due to tag stripping.

Table 1: Multi-stage observation retention matrix across semantic metadata channels in `browser-use 0.13.10`.

Metadata Channel	Stage 1 (HTML)	Stage 2 (Live DOM)	Stage 3 (AXTree)	Stage 4 (Serializer)	Stage 5 (Prompt)	Architectural Filtering Mechanism
Schema.org JSON-LD	Retained	Retained	Dropped	Dropped	Dropped	Blacklisted via `DISABLED_ELEMENTS` (L18)
Button WAI-ARIA	Retained	Retained	Retained	Retained	Retained	Whitelisted on interactive elements
Non-Interactive ARIA	Retained	Retained	Retained	Dropped	Dropped	Non-interactive tags stripped (L1046)
Custom `data-*` (Default)	Retained	Retained	Dropped	Dropped	Dropped	Omitted from default attribute whitelist
Custom `data-*` (Extended)	Retained	Retained	Dropped	Retained	Retained	Delivered on buttons under extended whitelist
Temporal `<meta>`	Retained	Retained	Dropped	Dropped	Dropped	Blacklisted via `DISABLED_ELEMENTS` (L18)
Temporal `<time>`	Retained	Retained	Retained	Stripped	Retained	Tag stripped; inner text emitted

4 Empirical Systems Audit: The 5-Stage Observation Pipeline

4.1 The 5 Observation Pipeline Stages

As illustrated in Figure 1, web information flows through distinct representations:

1. **Stage 1 (Source HTML & Static Markup):** Raw HTML markup received over HTTP.
2. **Stage 2 (Browser Live DOM):** In-memory DOM tree parsed by Blink and augmented by client-side JavaScript.
3. **Stage 3 (Browser Accessibility Tree):** Chromium Blink native `AXObjectCache` queried via CDP `Accessibility.getFullAXTree`. This operates in parallel with DOM serialization.
4. **Stage 4 (Agent DOM Serializer):** The simplified selector map and pruned tree constructed by `browser-use 0.13.10`.
5. **Stage 5 (Final Model Context Prompt):** The stringified observation buffer compiled into the LLM context.

Importantly, Stage 3 is not a sequential intermediary between Stage 2 and Stage 4. In `browser-use 0.13.10`, the serializer walks the live DOM directly (Stage 2 $\rightarrow$ Stage 4). Chromium Blink independently constructs its native Accessibility Tree (Stage 2 $\rightarrow$ Stage 3).

4.2 Multi-Stage Observation Retention Matrix

We executed our automated verification harness (`audit_metadata_reachability.py`) across benchmark variants and edge-case fixtures (`fixtures/edge_cases.html`), intercepting prompt payloads using a local `MockChatModel`. Table 1 reports deterministic retention status across pipeline stages.

4.3 Node-Level Correspondence: Chromium AXTree vs. Selector Map

To evaluate structural alignment between Chromium's native accessibility tree and the agent's observation tree, we implemented `compare_accessibility_tree.py`:

- CDP `Accessibility.getFullAXTree` extracts Blink's unique `backendDOMNodeId` for accessibility nodes.
- In `browser-use 0.13.10`, `summary.dom_state.selector_map` indexes interactive elements containing the matching Blink `backend_node_id`.

Empirical Findings on Route `#home`: Across 336 Chromium AXNodes constructed across the page (256 possessing a discrete `backendDOMNodeId`), `browser-use` registered 34 interactive elements in its selector map. Exactly **27 of 34 interactive elements (**79.4%**)** matched 1-to-1 to a corresponding Blink AXNode.

Forensic extraction of the 7 unmatched elements reveals that all 7 are stock-status badge spans across the 7 product cards:

```
<span class="product-stock in-stock">In Stock</span>
```

The agent framework registered them as clickable (assigning selector indices 69, 84, 99, 114, 129, 144, 159) due to client-side heuristics (bounding box and click listeners). In contrast, Chromium Blink's `AXObjectCache` flattened them into inline text runs within parent cards, marking them with `ignored: true` and omitting discrete top-level accessibility objects.

Estimand and Inferential Scope: We emphasize that these 34 interactive elements represent a complete, deterministic census of route `#home` under the audited browser configuration, not a random sample. Consequently, the descriptive proportion ($27/34 = 79.4\%$) is the primary finding. The Wilson 95% confidence interval ($[63.2\%, 89.7\%]$) reflects hypothetical sampling variance under a superpopulation model generalizing across similar e-commerce layouts, rather than measurement uncertainty for the observed route.

4.4 Structural Divergence Between DOM Serialization and Native Accessibility Trees

Our comparative analysis reveals bidirectional divergence between native accessibility trees and serialized DOM observations:

- **Non-Interactive ARIA Semantics:** Retained by Chromium Blink's AXTree, but stripped by `browser-use 0.13.10`. In our stimulus markup (`variants/v2-aria/index.html`), we examine two distinct patterns:
  1. Price Span: `<span class="product-price" aria-label="Price: $49.99">$49.99</span>`. Here, the element lacks an explicit ARIA role. Under W3C WAI-ARIA 1.2 [14] §5.2.7 and the Accessible Name and Description Computation (AccName 1.1) specification [13], elements with `role="generic"` prohibit author-supplied naming; standard screen readers ignore `aria-label` on generic spans. Nonetheless, Chromium Blink's internal `AXObjectCache` constructs an accessibility node with computed name `"Price: $49.99"`. In contrast, `browser-use 0.13.10` strips the tag entirely at line 1046, delivering only the text `"$49.99"`.
  2. Rating Span: `<span class="product-rating" role="img" aria-label="Rating: 4.5 out of 5 stars">...</span>`. Here, the explicit `role="img"` is a valid naming container under ARIA 1.2. Blink retains the accessible name, but `browser-use` strips the element because it is non-interactive.
- **Custom Developer `data-*` Attributes:** Completely absent from Blink's accessibility tree (custom attributes have no accessibility semantics), but deliverable to the agent on interactive elements when `include_attributes` is extended.

5 Methodological Analysis and Experimental Confounds

5.1 The Action-Label Co-Location Confound

When ARIA markup does reach the prompt (on interactive `<button>` elements), augmenting button labels from generic text (e.g., `"Add to Cart"`) to entity-specific ARIA labels (e.g., `aria-label="Add SoundFlow Air 3 to cart"`) introduces an acute experimental confound:

- In unaugmented baseline (ABS): The agent observes:

  ```
  SoundFlow Air 3
  $49.99
  [12] <button>Add to Cart</button>
  ```

  To select the correct product, the agent must perform spatial or structural reference resolution, associating the preceding typography with button `[12]`.

- In entity-augmented ARIA (ARIA): The agent observes:

  ```
  [12] <button aria-label="Add SoundFlow Air 3 to cart">Add to Cart</button>
  ```

  The target product name is now co-located directly inside the clickable element's attributes.

We evaluate generic versus entity-specific ARIA labels as distinct conceptual factors. Entity-specific labels can provide genuine disambiguation benefits in dense interfaces, resolving reference ambiguity. However, in experimental ablation studies intended to evaluate the utility of semantic accessibility channels, injecting entity identifiers directly into button labels confounds the manipulation: it replaces multi-node spatial and semantic reasoning with trivial string matching between the task instruction and the interactive element's accessible name.

5.2 Hardened State-Asserted Verifiers

In empirical agent benchmarking, relying on self-reported agent text (e.g., "I have added the product to the cart") introduces severe measurement error. In our testing harness, all task completions are verified exclusively through deterministic queries against application state:

- Direct inspection of backend SQLite database state (e.g., verifying order rows).
- Inspection of browser client-side storage (`localStorage.cartState`).
- CDP DOM node attribute verification (e.g., verifying active cart item count attributes).

5.3 Exploratory Cloud Pilot: The Ceiling Effect

We executed an exploratory cloud pilot across 41 completed trials using Gemini 2.5 Flash Lite via Google AI Studio ('google-genai 1.55.0') across three experimental arms (13 ABS, 13 ARIA, 15 DATA-CFG) on simple e-commerce tasks. All 41 completed runs achieved task success (100.0%, Wilson 95% CI: $[91.4\%, 100.0\%]$; ABS: 13/13 $[77.2\%, 100.0\%]$, ARIA: 13/13 $[77.2\%, 100.0\%]$, DATA-CFG: 15/15 $[79.6\%, 100.0\%]$). The observed pairwise risk differences are identically zero ($\hat{\theta} = 0.0$, Newcombe 95% CI: $[-0.228, +0.228]$). In denominator accounting, 44 total execution attempts were recorded; 3 attempts in Run 26 encountered transient upstream `503 UNAVAILABLE` network errors and were successfully retried on attempt 4 under the framework's retry policy without data corruption.

Critically, this 100% completion rate represents an acute **task-battery ceiling effect**: on simple single-product addition and linear form completion tasks, visible typography alone was completely sufficient for task success. Because sample variance was identically zero across all arms, this pilot demonstrates the operational execution stability of the harness, but it **does not demonstrate behavioral equivalence or treatment efficacy** between metadata formats.

6 Statistical Power Boundaries and Equivalence Analysis

6.1 Task-Stratified CMH and Newcombe Hybrid Score Intervals

To analyze multi-arm agent experiments without assuming normal approximations near probability boundaries, we implemented exact statistical estimators in pure Python:

- **Cochran–Mantel–Haenszel (CMH) Test:** Tests conditional independence across task strata $k = 1, \ldots, K$. Audited against the Agresti (2002) [1] multicenter clinical benchmark ($K = 8$), our implementation yields $\chi^2 = 6.3841, p = 0.01151$, matching published figures to 4 decimal places.
- **Newcombe Hybrid Score Interval (Method 10):** Constructs closed-form risk difference intervals by combining Wilson bounds. Audited against Newcombe (1998) [11] (56/70 vs. 38/80), our implementation yields $\hat{\theta} = 0.3250$, 95% CI: $[+0.1764, +0.4578]$, matching published benchmarks exactly.

Multiple Comparisons Adjustment: To control family-wise error rates, Bonferroni correction must distinguish comparison families:

- In a 3-arm design (ABS, ARIA, DATA-CFG), there are $\binom{3}{2} = 3$ pairwise comparisons, setting nominal significance to $\alpha = 0.05/3 \approx 0.0167$.
- When evaluating unpooled exploratory sensitivity contrasts across 5 individual tasks, significance is set to $\alpha = 0.05/5 = 0.0100$.

6.2 Monte Carlo Power Simulations

Using a Randomized Complete Block Design (RCBD) across 5 tasks with task-level variance $\sigma^2 = 0.09$ (logit scale), we executed 10,000-trial Monte Carlo power simulations (random seed 42) in 'power_analysis.py' ($n = 20$ per arm, $\alpha = 0.0167$):

- Large Disparity ($\Delta = 0.70$, $p_0 = 0.85 \to p_1 = 0.15$): Power $= 98.6\% \pm 0.3\%$.
- Moderate Effect ($\Delta = 0.35$, $p_0 = 0.80 \to p_1 = 0.45$): Power $= 49.6\% \pm 1.1\%$.
- Small Effect ($\Delta = 0.20$, $p_0 = 0.80 \to p_1 = 0.60$): Power $= \mathbf{10.8}\% \pm \mathbf{0.6}\%$.

These effect sizes serve as illustrative sensitivity scenarios: at small sample sizes ($n = 20$), power to detect a realistic 20-percentage-point difference is only 10.8%. Over 89% of such trials will yield a false negative ($p > 0.05$).

6.3 Sample Size Requirements for TOST Equivalence

To demonstrate equivalence between formats within margin $(-\delta_{\text{equiv}}, +\delta_{\text{equiv}})$, researchers must apply Two One-Sided Tests (TOST):

$$n = \left(\frac{(z_{1-\alpha} + z_{(1+\text{power})/2}) \cdot \sqrt{2 \cdot p \cdot (1-p)}}{\delta_{\text{equiv}}} \right)^2 \tag{1}$$

For $\alpha = 0.05$ (one-sided) and 80% power, $z_{\text{crit}} = 1.64485 + 1.28155 = 2.92640$. For an illustrative equivalence margin of $\delta_{\text{equiv}} = \pm 0.10$ (± 10 percentage points):

- At $p = 0.70$: Establishing equivalence within ± 0.10 requires $n = 359.8 \approx 360$ trials per arm ($N = 1,080$ for 3 arms, or $N = 1,440$ across 4 arms).

- At $p = 0.50$ (maximum variance): Required sample size rises to $n = 428.3 \approx 429$ per arm ($N = 1,287$ for 3 arms).
- Clustered Design Effect: If repeated trials within tasks induce clustering ($\rho = 0.10, m = 4 \implies \text{DEFF} = 1+(m-1)\rho = 1.30$), required sample size increases to $n = 360 \times 1.30 \approx 468$ per arm ($N = 1,404$ for 3 arms).

We emphasize that an equivalence margin of ± 0.10 is illustrative; it is not an empirically established web-usability tolerance standard. Claiming equivalence from small-sample non-significant results ($n = 13$ to 20) constitutes **The Equivalence Fallacy**.

7 Discussion: Architectural Trade-Offs and Implications

7.1 Benefits and Risks of Observation Pruning

Our findings highlight the fundamental systems tension in browser-agent observation design:

- **Pruning Benefits:** Raw web pages frequently exceed 20,000 DOM nodes and 100,000 tokens. Heuristic pruning dramatically compresses the input context, reduces inference cost, prevents context window exhaustion, and removes noisy subtree boilerplate. Furthermore, stripping non-rendered `<script>` and `<style>` tags inadvertently filters out hidden script-embedded text.
- **Pruning Risks:** Overly aggressive pruning discards structured semantic metadata (Schema.org JSON-LD, container ARIA labels, custom attributes) that web authors specifically provide to clarify page meaning. When tags are stripped, semantic relationships between containers and interactive children are lost.

7.2 Implications for Web Standards and Out-of-Band Interfaces

Proposals to enrich human accessibility trees (WAI-ARIA) specifically for AI scrapers face dual obstacles: first, client-side serializers often strip non-interactive ARIA annotations; second, overloading ARIA with redundant machine metadata risks degrading accessibility tree navigation for human screen-reader users. Consequently, emerging out-of-band protocols—such as `llms.txt` [8] and the Model Context Protocol (MCP) [2]—offer a cleaner separation of concerns by delivering machine-readable context directly to agents without relying on DOM serialization heuristics.

8 Limitations

We explicitly identify several key limitations:

1. **Single-Framework Scope:** All AST audits and serialization measurements were conducted on `browser-use 0.13.10`. Other frameworks (e.g., BrowserGym, Stagehand, Skyvern, Crawl4AI) employ different observation pipelines, accessibility snapshots, or raw HTML distillation heuristics.
2. **Observation Modality:** Audits were performed within a text-only observation space (`use_vision=False`). Multimodal agents utilizing screenshot inputs can perceive rendered typography and visual styling directly, bypassing DOM serializer pruning.
3. **Synthetic Evaluation Environment:** Measurements utilized a controlled e-commerce benchmark environment (`variants/v0`–`v3`). While this isolates markup channels from visual occlusions, real-world websites feature dynamic scripts and layout variations not captured in synthetic fixtures.

4. **Ceiling Effect and Unexecuted Behavioral Trials:** The 41-trial cloud pilot exhibited a ceiling effect (100% success across all arms). Demonstrating whether metadata channel reachability translates into statistically significant behavioral differences requires a future channel-isolation experiment (specified in our repository protocol, requiring $N \geq 1,080$ trials for equivalence).

5. **Finite Census Scope:** The node correspondence comparison was evaluated on route `#home` ($n = 34$ interactive elements), producing a finite descriptive census rather than a broad multi-site sample.

9 Conclusion

This paper presents an empirical systems audit of the DOM serialization gap in autonomous browser agents, establishing that default observation pipelines in `browser-use 0.13.10` deterministically prune structured metadata (Schema.org JSON-LD and custom developer attributes) and strip non-interactive container ARIA annotations prior to model prompt construction. We have documented structural divergence between native browser accessibility trees and DOM serializers, formalized the Action-Label Co-Location Confound, and established analytical sample-size boundaries for web agent benchmarks. By providing an entirely reproducible offline verification suite, this work establishes essential systems baselines for future web agent architecture and evaluation.

References

[1] Alan Agresti. *Categorical Data Analysis*. John Wiley & Sons, Hoboken, NJ, 2nd edition, 2002.

[2] Anthropic. The model context protocol: An open standard for connecting AI models to data sources. https://modelcontextprotocol.io/, 2024.

[3] Léo Boisvert, Massimo Caccia, Megh Thakkar, Alexandre Drouin, Nicolas Chapados, and Alexandre Lacoste. WorkArena++: Towards enterprise-grade autonomous web agents. *arXiv preprint arXiv:2407.05291*, 2024.

[4] Xiang Deng, Yu Gu, Boyuan Zheng, Shijie Chen, Samuel Stevens, Boshi Wang, Huan Sun, and Yu Su. Mind2Web: Towards a generalist agent for the web. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

[5] Alexandre Drouin, Maxime Gasse, Massimo Caccia, Issam H. Laradji, Manuel Del Verme, Tom Marty, Léo Boisvert, Megh Thakkar, Quentin Cappart, David Vazquez, Nicolas Chapados, and Alexandre Lacoste. WorkArena: How capable are web agents at solving common knowledge work tasks? In *International Conference on Machine Learning (ICML)*, 2024.

[6] Izzeddin Gür, Hiroki Furuta, Austin Huang, Mustafa Safdari, Yutaka Matsuo, Douglas Eck, and Aleksandra Faust. A real-world WebAgent with planning, long context understanding, and program synthesis. In *International Conference on Learning Representations (ICLR)*, 2024.

[7] Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. WebVoyager: Building an end-to-end web agent with large multimodal models. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[8] Jeremy Howard. The `/llms.txt` file format for standardizing machine-readable context for language models. https://llmstxt.org/, 2024.

[9] Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Chong Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Ruslan Salakhutdinov, and Daniel Fried. VisualWebArena: Evaluating multimodal web agents on realistic visual web tasks. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[10] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations (ICLR)*, 2024.

[11] Robert G. Newcombe. Interval estimation for the difference between independent proportions: comparison of eleven methods. *Statistics in Medicine*, 17(8):873–890, 1998.

[12] Schema.org Community Group. Schema.org: Structured data for the web. https://schema.org/, 2024.

[13] World Wide Web Consortium. Accessible name and description computation 1.1. W3c recommendation, W3C, 2018.

[14] World Wide Web Consortium. Accessible rich internet applications (WAI-ARIA) 1.2. W3c recommendation, W3C, 2023.

[15] Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking multimodal agents for open-domain OS tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[16] Ke Yang, Yao Liu, Sapana Chaudhary, Rasool Fakoor, Pratik A. Chaudhari, George Karypis, and Huzefa Rangwala. AgentOccam: A simple yet strong baseline for LLM-based web agents. In *International Conference on Learning Representations (ICLR)*, 2025.

[17] Boyuan Zheng, Boyu Gou, Jihyung Kil, Huan Sun, and Yu Su. SeeAct: GPT-4V(ision) is a generalist web agent, if grounded. In *International Conference on Machine Learning (ICML)*, 2024.

[18] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous language agents. In *International Conference on Learning Representations (ICLR)*, 2024.

A Offline Statistical Pipeline Verification Benchmark

Synthetic Software Benchmark Disclaimer: The metrics in Tables 2 and 3 are generated from a mock/synthetic benchmark dataset (`test_scratch_synthetic_data/dataset_1_valid_complete.csv`) to verify that `analyze_v3_results.py` executes without numerical or serialization errors. **They do not represent real agent task performance.**

B Exploratory Local Hardware Diagnostics & Cancellation Telemetry

To inspect execution dynamics on constrained local hardware, we conducted five exploratory diagnostic runs on local quantized `qwen2.5:7b-instruct-q4_K_M` via Ollama on hybrid CPU/GPU

Table 2: Synthetic software pipeline output: Unpooled sensitivity contrasts across baseline arms (Tasks T1–T5, Bonferroni $\alpha = 0.0100$ for 5 contrasts). Pooling baseline conditions is rejected to prevent the Equivalence Fallacy in the absence of a formal TOST margin.

Contrast ID	**Treatment** (n/N, %)	**Reference** (n/N, %)	**Risk Diff** ($\hat{\theta}$)	**Newcombe 95% CI**
ARIA vs BASELINE (ABS)	14 / 20 (70.0%)	13 / 20 (65.0%)	**+0.0500**	[-0.2312, +0.3168]
ARIA vs BASELINE_REP (VIS)	14 / 20 (70.0%)	12 / 20 (60.0%)	**+0.1000**	[-0.1901, +0.3644]
DATA-CFG vs BASELINE (ABS)	17 / 20 (85.0%)	13 / 20 (65.0%)	**+0.2000**	[-0.0754, +0.4320]
DATA-CFG vs BASELINE_REP (VIS)	17 / 20 (85.0%)	12 / 20 (60.0%)	**+0.2500**	[-0.0267, +0.4795]
ARIA vs DATA-CFG	14 / 20 (70.0%)	17 / 20 (85.0%)	**-0.1500**	[-0.3751, +0.0906]

Table 3: Baseline stability and quality control check on synthetic benchmark data.

Condition	**Success** (n/N)	**Success Rate** (%)	**Baseline Difference** ($\hat{\theta}$ and 95% CI)	**Fisher's Exact Test** (p-value)
BASELINE (ABS)	13 / 20	65.0%	+0.0500	$p = 1.0000$
BASELINE_REP (VIS)	12 / 20	60.0%	95% CI: $[-0.2312, +0.3168]$	(two-tailed, unadjusted)

Methodological Decision on Baseline Pooling: A two-tailed Fisher's exact test fails to reject the null hypothesis of equal baseline performance ($p = 1.0000 \geq 0.05$). Under statistical equivalence principles, failure to detect a difference cannot be equated with evidence of equivalence (*The Equivalence Fallacy*). In the absence of an *a priori* prespecified Two One-Sided Tests (TOST) equivalence margin (δ_{equiv}), pooling the two baseline conditions is unjustified. Consequently, baseline conditions are evaluated and reported unpooled in all primary sensitivity contrasts (Table 2).

hardware (AMD Ryzen 5 5600H, 16 GB DDR4 RAM, NVIDIA GeForce RTX 3050 Laptop GPU with 4 GB VRAM). Table 4 reports the telemetry from `results/local_pilot_raw_logs.json`.

Telemetry Analysis and Harness Observations:

- **Watchdog Ceilings vs. Hardware Latency:** In run 003, with prompt expansion exceeding 3,200 tokens across multi-turn history, prompt ingestion and generation latency combined to exceed the 180-second per-step watchdog, triggering execution termination. Post-hoc verification confirmed that the cart interaction had completed prior to timeout.
- **Cancellation Telemetry Capture:** In Python 3.8+, `asyncio.CancelledError` inherits from `BaseException`. In run 004, standard `except Exception:` handlers failed to catch watchdog cancellations. In run 005, dedicated `except asyncio.CancelledError:` handling successfully logged the in-flight timeout without corrupting trial records.

These five exploratory runs illustrate harness engineering boundaries and watchdog behavior under local execution; they do not represent generalizable findings on model reasoning capabilities.

C In-Process Request Metering and Quota Governance Architecture

During experimental agent evaluation, unexpected multi-turn loops or runaway concurrency can exhaust API rate limits or incur unintended costs. In our testing harness, client-side requests are bounded via a thread-safe, fail-closed quota architecture implemented in `budget_controller.py`:

- **Pre-Dispatch Reservation:** A proxy model wrapper (`BudgetGuardedChatModel`) intercepts every invocation of `ainvoke()` immediately prior to network transmission. Under a reentrant thread lock (`threading.RLock`), it queries `check_can_dispatch_request()`. If cu-

Table 4: Exploratory diagnostic runs on local `qwen2.5:7b-instruct-q4_K_M` (Task T2).

Run ID	Arm	Variant	Duration	Steps	CDP State (Ground Truth)	Observed Mechanism
`qwen_diag_001`	DATA-CFG	`v3-data-attrs`	3.34s	0	False	Pydantic validator exception (`max_history_items` parameter constraint).
`qwen_diag_002`	DATA-CFG	`v3-data-attrs`	1013.09s	2	**True (1.0)**	Step 2 clicked button; Step 3 blocked until HTTP read timeout due to missing outer watchdog.
`qwen_diag_003`	DATA-CFG	`v3-data-attrs`	288.61s	2	**True (1.0)**	Step 1 succeeded (102.9s); Step 2 breached 180s watchdog ceiling.
`qwen_diag_004`	ABS	`v3-data-attrs`	185.50s	1	False	Step 1 watchdog timeout; unhandled cancellation dropped telemetry.
`qwen_diag_005`	ABS	`v3-data-attrs`	188.56s	1	False	Step 1 watchdog timeout; cancellation telemetry safely captured.

mulative requests meet or exceed the ceiling (e.g., 600 requests), it raises `BudgetExceededError`, failing closed.

- **Atomic Reservation Accounting:** Upon clearance, `record_request_pre_dispatch()` atomically increments the persistent request counter on disk before the network socket opens. Even if the subsequent HTTP request drops or times out, the attempt is accounted for against the ceiling.
- **Post-Dispatch Token Reconciliation:** Following successful completion, `record_request_post_dispatc` reconciles prompt and completion token counts using provider-reported usage metadata, updating cost calculations without double-counting the request.
- **Trial Finalization Decoupling:** At trial completion, `record_trial_completion(requests_already_re` increments trial metrics without re-incrementing the request counter, guaranteeing exact 1-to-1 metering.

Scope and Operational Boundaries: This architecture guarantees strict request ceilings within a single Python runtime and concurrency model. It does not provide inter-process synchronization across multiple external script invocations, nor does it replace provider-side billing console caps.

D Step-by-Step Offline Reproducibility Instructions

All systems findings and statistical bounds reported in this paper can be reproduced offline on Windows or Linux environments in under two minutes without network access or live API calls:

1. **Full Automated Test Suite (126 Tests):** Verify all 126 regression and integration tests across quota controls, verifiers, and pipeline stages (100% passing in $\sim$ 15s):

```
python -m unittest discover -s . -p "test_*.py"
```

2. **Multi-Channel Reachability Trace (Table 1):** Trace metadata retention across the 5-stage observation pipeline to generate `serialization_audit_metrics.json`:

```
python audit_metadata_reachability.py
```

3. **Chromium Blink AXTree vs. Serializer Comparison (Section 4):** Extract Blink accessibility nodes via CDP and evaluate correspondence against `browser-use`:

```
python compare_accessibility_tree.py
```

4. **TOST Power Bounds Reproduction (Section 6.3):** Compute required sample sizes for statistical equivalence testing under varying margins ($N \geq 1,440$ across 4 arms, $n \approx 360$/arm at $p = 0.70$, $\delta = \pm 0.10$):

```
python reproduce_power_audit.py
```

5. **Statistical Engine Clinical Benchmark (Appendix A):** Validate Newcombe hybrid score CIs, Cochran-Mantel-Haenszel tests, and Fisher's exact tests against clinical reference datasets:

```
python -m unittest test_analyze_v3_results.py
```